

DOCUMENTACIÓN TÉCNICA · TP-TACS-2026-GRUPO-4

Deploy en Google Cloud Run

Guía de compatibilidad, cambios realizados y pasos de despliegue para Figuritas.Api y Figuritas.Client en un entorno serverless escalable.

PROYECTO

tp-tacs-2026-grupo-4

PLATAFORMA OBJETIVO

Google Cloud Run (serverless)

STACK

ASP.NET Core 10 + Blazor WASM + MongoDB

FECHA

Junio 2026

SECCIÓN 1

Resumen: ¿Es compatible con Cloud Run?

VEREDICTO

Sí, ambas aplicaciones son compatibles con Google Cloud Run sin necesidad de reescribir la arquitectura. Hay ajustes puntuales — todos aplicados en este documento — y dos decisiones de infraestructura (dónde corre MongoDB y un worker en background).

El punto más importante para el equipo era la **autenticación**. La buena noticia: es el aspecto *más fuerte* del diseño para serverless. La autenticación basada en **JWT Bearer** es 100% stateless: cualquier instancia de la API puede validar un token emitido por cualquier otra instancia, sin sesiones en memoria, sin cookies de sesión, sin bases de datos de sesión. Escalar horizontalmente de 1 a N instancias no requiere ningún cambio en el flujo de autenticación.

Área	Estado	Detalle
JWT / Autenticación	Listo	100% stateless. Solo requiere mismo <code>Jwt__Key</code> en todas las instancias.
Client (Blazor WASM)	Listo	SPA estática servida por nginx. Cold start casi nulo.
Nginx → API proxy	Listo	Parametrizado con <code>API_BASE_URL</code> (cambio aplicado).
HTTPS redirect	Listo	Eliminado de la API (Cloud Run termina TLS en el borde).
MongoDB	Requiere acción	Debe hostearse externamente (MongoDB Atlas recomendado).
AuctionEndingWorker	Decisión pendiente	BackgroundService se pausa si el contenedor escala a cero. Ver sección 6.
SignalR	Limitación menor	Best-effort by design; degrada con gracia en multi-instancia.

SECCIÓN 2

Arquitectura de la solución

Los 3 modos de ejecución

La solución debe funcionar en tres contextos distintos sin conflictos entre sí:

Modo	Cómo se inicia	Cómo resuelve la URL de la API
Dev local	<code>dotnet run / dotnet watch</code>	<code>http://localhost:8080</code> — hardcodeado en <code>IsDevelopment()</code> branch de <code>Client/Program.cs</code> . No cambia.
Docker Compose	<code>docker-compose up</code>	nginx hace <code>proxy_pass</code> a <code>http://web-api:8080</code> (DNS interno de Docker). Configurado via variable <code>API_BASE_URL=http://web-api:8080</code> .
Cloud Run	Dos servicios Cloud Run separados	nginx hace <code>proxy_pass</code> a la URL pública del servicio API. Configurado via variable <code>API_BASE_URL=https://figuritas-api-xxxx.run.app</code> .

Flujo de una petición autenticada

Modo Docker Compose / Cloud Run (browser): [Browser] | GET /api/stickers (+ Authorization: Bearer {jwt}) ▼

[nginx – Figuritas.Client] | proxy_pass \${API_BASE_URL}/api/stickers ▼ [Figuritas.Api] | Valida JWT con Jwt:Key (env var, mismo valor en todas las instancias) | Consulta MongoDB ▼ [Respuesta 200 JSON] ↪ nginx

→ Browser **Modo dev local (dotnet run):** [Browser / Blazor WASM] | GET http://localhost:8080/api/stickers | (apiBaseAddress hardcodeado en IsDevelopment()) ▼ [Figuritas.Api en :8080]

En todos los modos, el token JWT viaja en el **header Authorization**, no en una cookie. Esto es clave: no hay problemas de SameSite, cross-domain cookies ni sticky sessions para auth.

SECCIÓN 3

Cambios realizados

Todos los cambios listados a continuación fueron aplicados en el repositorio. Se muestran el estado anterior y el nuevo para cada archivo.

3.1 — Figuritas.Client/nginx.conf

Se convierte en un **template parametrizable**. La imagen oficial `nginx:alpine` procesa automáticamente los archivos en `/etc/nginx/templates/`: sustituye variables de entorno con `envsubst` y genera el `.conf` final antes de iniciar nginx.

Antes

```
# proxy_pass apuntaba al nombre fijo del servicio Docker
location /api/ {
    proxy_pass http://web-api:8080; # ← no funciona fuera de Docker Compose
    proxy_set_header Host $host;   # ← host del cliente, no del upstream
}
```

Después

```
# ${API_BASE_URL} se inyecta como variable de entorno al arrancar el contenedor
location /api/ {
    proxy_pass ${API_BASE_URL};          # http://web-api:8080 ó https://figuritas-api-xxxx.run.app
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection keep-alive;
    proxy_set_header Host $proxy_host;  # ← host del upstream (correcto para TLS/SNI)
    proxy_ssl_server_name on;           # necesario para SNI cuando upstream es HTTPS
    proxy_cache_bypass $http_upgrade;
}
```

La variable `NGINX_ENVSUBST_FILTER=^API_BASE_URL$` (seteada en docker-compose) le indica a nginx que *solo* sustituya `API_BASE_URL`, dejando intactas las variables internas de nginx (`$http_upgrade` , `$proxy_host` , etc.).

3.2 — Dockerfile (target client-runtime)

Se cambia el destino del `COPY` de nginx.conf: en vez de copiarlo directamente a `/etc/nginx/conf.d/default.conf` (estático), va a `/etc/nginx/templates/` para que nginx:alpine lo procese con `envsubst` al iniciar.

Antes

```
COPY Figuritas.Client/nginx.conf /etc/nginx/conf.d/default.conf
```

Después

```
COPY Figuritas.Client/nginx.conf /etc/nginx/templates/default.conf.template
```

3.3 — docker-compose.yml y docker-compose.prod.yml

Se agrega la variable de entorno `API_BASE_URL` al servicio `web-client` para que nginx sepa a qué contenedor de API redirigir.

```
web-client:
  environment:
    - API_BASE_URL=http://web-api:8080      # mismo valor que antes, ahora explícito
    - NGINX_ENVSUBST_FILTER=^API_BASE_URL$$ # solo sustituye esta variable
```

El comportamiento de Docker Compose no cambia: `web-api:8080` sigue resolviendo via DNS interno de Docker.

3.4 — Figuritas.Api/Program.cs

Se elimina `app.UseHttpsRedirection()`. En Cloud Run, Google Front End termina el TLS y reenvía tráfico HTTP al contenedor en el puerto 8080. Este middleware no tiene efecto útil (Kestrel no tiene binding HTTPS configurado) y solo generaba logs de warning innecesarios.

Eliminado

```
// Antes: esta línea existía entre app.Build() y app.UseAuthentication()
app.UseHttpsRedirection(); // ← eliminado
```

Cloud Run ya garantiza HTTPS en la capa de borde. El contenedor puede correr HTTP puro en 8080 sin exponer HTTP a internet.

SECCIÓN 4

Variables de entorno por entorno

Figuritas.Api

Variable	Dev local	Docker Compose	Cloud Run
Jwt__Key	dotnet user-secrets	\${JWT_KEY} (desde .env)	Secret Manager → env var en el servicio
Mongo__ConnectionString	mongodb://localhost:27017 (appsettings)	mongodb://mongo- tacs:27017 (compose)	URI de MongoDB Atlas (ej. mongodb+srv://user:pass@cluster.mongodb.net)
Mongo__DatabaseName	FiguriTACS_DB	FiguriTACS_DB	FiguriTACS_DB (o el que se elija)
AllowedOrigins	No requerida (appsettings trae defaults localhost)	No requerida (proxy nginx evita CORS cross-origin)	Opcional: URL del Client si se hacen llamadas directas fuera del proxy
ASPNETCORE_ENVIRONMENT	Automático (Development)	Production en prod-compose	Production

Figuritas.Client (nginx)

Variable	Dev local	Docker Compose	Cloud Run
API_BASE_URL	No aplica (nginx no corre)	http://web-api:8080	https://figuritas-api-xxxx.run.app
NGINX_ENVSUBST_FILTER	No aplica	^API_BASE_URL\$	^API_BASE_URL\$

IMPORTANTE — JWT__KEY EN PRODUCCIÓN

El secreto JWT debe ser una cadena aleatoria de al menos 32 caracteres y **nunca** commitearse al repositorio. En Cloud Run, guardarlo en **Google Secret Manager** y montarlo como variable de entorno en la definición del servicio. Todas las revisiones/instancias del servicio Api deben usar el mismo secreto para poder validar tokens entre sí.

SECCIÓN 5

Guía de deploy paso a paso en Cloud Run

PREREQUISITOS

Google Cloud SDK (`gcloud`) instalado y autenticado. Proyecto GCP creado. APIs habilitadas: Cloud Run, Cloud Build, Artifact Registry, Secret Manager.

5.1 — MongoDB Atlas (base de datos)

- 1 Crear cuenta en `mongodb.com/cloud/atlas` → New Project → Create Cluster (tier `M0 Free` alcanza para el TP).
- 2 En `Database Access` : crear usuario con contraseña segura (ej. `figuritas-prod`).
- 3 En `Network` : `0.0.0.0/0` (Allow from anywhere). Cloud Run no tiene IP fija de salida; esta es la solución pragmática para el TP. En producción real se configuraría un Serverless VPC Connector + Cloud NAT para IP estática.
Access agregar IP
- 4 Copiar la URI de conexión: `mongodb+srv://figuritas-prod:PASSWORD@cluster0.xxxxx.mongodb.net/FiguriTACS_DB` .

5.2 — Guardar secretos en Secret Manager

```
# Habilitar La API
gcloud services enable secretmanager.googleapis.com

# Crear secreto para JWT
echo -n "TuClaveSecretaDeAl_Menos_32_Caracteres_Prod" | \
  gcloud secrets create figuritas-jwt-key --data-file=-

# Crear secreto para MongoDB
echo -n "mongodb+srv://figuritas-prod:PASS@cluster0.xxx.mongodb.net/FiguriTACS_DB" | \
  gcloud secrets create figuritas-mongo-uri --data-file=-
```

5.3 — Subir imágenes a Artifact Registry

```
# Crear repositorio Docker en Artifact Registry
gcloud artifacts repositories create figuritas \
  --repository-format=docker \
  --location=us-central1

# Autenticar Docker con GCP
gcloud auth configure-docker us-central1-docker.pkg.dev

# Build y push de la API
docker build --target api-runtime \
  -t us-central1-docker.pkg.dev/TU_PROYECTO/figuritas/api:latest .
docker push us-central1-docker.pkg.dev/TU_PROYECTO/figuritas/api:latest

# Build y push del Client (primero deployar la API para conocer su URL)
docker build --target client-runtime \
  -t us-central1-docker.pkg.dev/TU_PROYECTO/figuritas/client:latest .
docker push us-central1-docker.pkg.dev/TU_PROYECTO/figuritas/client:latest
```

5.4 — Deployar Figuritas.Api

```
gcloud run deploy figuritas-api \
  --image=us-central1-docker.pkg.dev/TU_PROYECTO/figuritas/api:latest \
  --region=us-central1 \
  --platform=managed \
  --allow-unauthenticated \           # La API es pública (consumida por browsers con JWT)
  --port=8080 \
  --min-instances=1 \                # necesario para el AuctionEndingWorker (ver sección 6)
  --max-instances=5 \
  --set-secrets="Jwt__Key=figuritas-jwt-key:latest,\
Mongo__ConnectionString=figuritas-mongo-uri:latest" \
  --set-env-vars="Mongo__DatabaseName=FiguriTACS_DB,\
ASPNETCORE_ENVIRONMENT=Production"
```

ANOTAR LA URL DEL SERVICIO API

Al finalizar el deploy, `gcloud run` muestra la URL del servicio:

Service URL: `https://figuritas-api-xxxx-uc.a.run.app`

Esta URL se usa en el siguiente paso como `API_BASE_URL` para el Client.

5.5 — Deployar Figuritas.Client

```
gcloud run deploy figuritas-client \
  --image=us-central1-docker.pkg.dev/TU_PROYECTO/figuritas/client:latest \
  --region=us-central1 \
  --platform=managed \
  --allow-unauthenticated \
  --port=80 \
  --set-env-vars="API_BASE_URL=https://figuritas-api-xxxx-uc.a.run.app,\
NGINX_ENVSUBST_FILTER=^API_BASE_URL$$"
```


5.6 — Verificar el deploy

- 1 Abrir la URL del Client en un browser (ej. `https://figuritas-client-xxxx.run.app`).
- 2 Intentar login con un usuario semilla (ej. `FiguCollector1 / Pass2026`).
- 3 Verificar en Network DevTools que las `/api/...` devuelven 200 y que el `Authorization: Bearer ...` está presente.
- 4 Verificar en Cloud Run Console → Logs que no aparecen errores de MongoDB ni de configuración JWT.

SECCIÓN 6

Consideraciones avanzadas

6.1 — AuctionEndingWorker y scale-to-zero

El worker (`Figuritas.Api/Workers/AuctionEndingWorker.cs`) es un `BackgroundService` que hace polling cada 5 minutos para cerrar subastas expiradas y enviar notificaciones. Cloud Run, por defecto, puede escalar a cero instancias cuando no hay tráfico — en ese caso el worker no corre.

IMPACTO

Si el servicio Api escala a cero (sin tráfico entrante), las subastas no se cierran automáticamente hasta que llegue el próximo request que despierte el contenedor.

Opción A — Mínimo cambio (recomendada para el TP): ya aplicada en el deploy del paso 5.4 con `--min-instances=1` . Un contenedor queda siempre activo, el worker corre continuamente. El servicio sigue pudiendo escalar *hacia arriba* bajo carga (escalabilidad horizontal preservada). Solo se pierde la optimización de "cero instancias".

Opción B — Totalmente serverless (recomendada a largo plazo): Mover la lógica del worker a un endpoint HTTP interno (ej. `POST /internal/process-auctions`) y dispararlo cada 5 min con `Cloud Scheduler`. El código ya tiene soporte nativo para este patrón: los métodos `TryClaimEndingNotificationAsync` y `TryClaimAutomaticClosureAsync` usan *atomic claims* en MongoDB que garantizan idempotencia — múltiples instancias invocadas simultáneamente no generan duplicados.

6.2 — SignalR en multi-instancia

La app usa SignalR para notificaciones en tiempo real (`/api/notification-hub`). Con más de una instancia corriendo, un usuario conectado a la instancia A no recibirá en tiempo real un evento generado en la instancia B (no hay backplane compartido).

Esto **no es un bug** en el contexto actual: `NotificationService` ya trata SignalR como *best-effort* — las notificaciones siempre se persisten en MongoDB y están disponibles vía REST. El usuario las verá al refrescar o en la próxima carga de página.

Si se requiere real-time estricto entre instancias: agregar un backplane Redis con `builder.Services.AddSignalR().AddStackExchangeRedis("...")` usando Cloud Memorystore.

6.3 — CORS y el patrón de proxy nginx

Con el patrón actual (nginx hace proxy de `/api/*`), el browser nunca realiza peticiones *cross-origin*: desde su perspectiva, todo va al mismo origen (la URL del Client). Por eso **CORS no es un bloqueante** en esta configuración.

La variable `AllowedOrigins` en la API se vuelve relevante solo si en el futuro se hacen peticiones directas desde otro origen (ej. desde Swagger en un dominio distinto, o desde una app mobile). En ese caso, setear:

```
# Ejemplo: agregar AllowedOrigins al deploy de la API
--set-env-vars="AllowedOrigins=https://figuritas-client-xxxx.run.app"
```

6.4 — Notas sobre Docker Compose (modos existentes)

- Dev (`docker-compose up`): sin cambios en el comportamiento. La API está disponible en `localhost:5000` y el Client en `localhost:5001` .

- Prod local (`docker-compose.prod.yml`): idem. Requiere que el archivo `.env` tenga `JWT_KEY` y (en esta versión) `Mongo__ConnectionString` apuntando a Atlas si se quiere testear con la DB de producción.
- Dev local (`dotnet run`): sin cambios. El branch `IsDevelopment()` en `Client/Program.cs` sigue apuntando a `http://localhost:8080` .